AeroNode

Autonomous testing runbook

Prepared 2 November 2026 | Windows / PowerShell | Scope: implemented Phases 1-3

Instructions to the testing agent

Test the existing AeroNode implementation completely within the scope below. Execute commands on the Windows machine containing C:\AeroNode; a chat session without local tool access cannot execute these tests. Read AGENTS.md if present, README.md, architecture.md, current_status.md and docs/phase2_validation.md and docs/phase3_validation.md before proceeding. Reconcile this guide with actual source and command help if the repository changes.

Run every required stage, preserve logs and generated evidence, and finish with a stage-by-stage PASS, FAIL or BLOCKED report. Do not mark a stage PASS from old documentation. Do not silently skip failures. Continue independent checks when a stage is blocked.

Testing is authorized; do not rewrite source, retrain models, replace accepted checkpoints, delete data, modify global dependencies, or implement missing features. Write new results to a unique output folder. Repair local PATH settings as described. If further repair requires changes, explain the exact blocker and proposed change.

What a complete test means today

Required: environment and artifact inventory; fresh C++ build; Phase-1 Python/C++ numerical parity; four automated model/export tests; full 3,925-image validation of the accepted checkpoint; fresh four-output ONNX export and numerical parity on ten class-diverse validation images; evidence review and final report.

Not implemented: four-exit C++ inference, adaptive early exit, ZeroMQ control updates, policy benchmarks, API and browser dashboard. Report these as NOT IMPLEMENTED, not failed tests. The current C++ executable accepts the tiny Phase-1 model only. Never use its latency as the trained image classifier latency.

Previously observed evidence - reference only

On 2 November 2026 the Phase-1 100-iteration check passed with maximum absolute difference 1.4901161193847656e-08, predicted class 1, mean 0.00354 ms and P95 0.0037 ms. All four automated tests passed. The full dataset evaluation and accepted-checkpoint export were documented previously; they were not rerun during that inspection.

1. Prepare and capture evidence

```
Set-Location C:\AeroNode
$env:PATH = "C:\msys64\ucrt64\bin;C:\AeroNode\.venv\Scripts;" + $env:PATH
$run = Join-Path (Get-Location) ("output\testing-" +
  (Get-Date -Format "yyyyMMdd-HHmmss"))
New-Item -ItemType Directory -Path $run | Out-Null
Start-Transcript -Path (Join-Path $run 'session.txt')
git status --short
git rev-parse HEAD
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m pip freeze |
  Out-File (Join-Path $run 'packages.txt')
Get-ChildItem .deps
Get-Item build\runtime\aeronode_runtime.exe
Get-Item model\artifacts\simple_model.onnx
Get-Item model\artifacts\phase2-imagenette-5ep\best_final_exit.pt
Get-Item model\artifacts\phase3\multi_exit_imagenette.onnx
```

Check $LASTEXITCODE immediately after every native command; nonzero means failure, even if PowerShell continues. For PowerShell file operations, treat missing-file errors as blockers. Record the initial Git state so existing changes are not mistaken for test changes. Keep this PowerShell session alive so $run and PATH persist.

2. Build the existing C++ runtime

```
.\.venv\Scripts\cmake.exe -S . -B build -G Ninja `
  -DCMAKE_CXX_COMPILER=C:/msys64/ucrt64/bin/g++.exe `
  -DONNXRUNTIME_ROOT=C:/AeroNode/.deps/onnxruntime-win-x64-1.30.0
.\.venv\Scripts\cmake.exe --build build
```

PASS: both commands return 0 and build/runtime/aeronode_runtime.exe exists. An up-to-date build is acceptable. If the existing build cache uses a different generator/compiler, use a new build-test directory with the same configure flags, then substitute its executable path in Stage 3. Preserve the existing build directory.

If .venv or SDK is missing, record BLOCKED. Recovery on a new machine: create a local virtual environment, install requirements-phase1.txt and requirements-phase2.txt, install a compatible C++17 compiler, and provision the official Windows x64 ONNX Runtime 1.30.0 SDK. Do not assume downloaded model/data artifacts are included in a Git clone.

3. Verify the complete Phase-1 inference path

```
.\.venv\Scripts\python.exe tests\verify_phase1.py `
  --executable build\runtime\aeronode_runtime.exe `
  --model model\artifacts\simple_model.onnx `
  --iterations 500
```

PASS: exit code 0, JSON status "verified", 500 iterations, class 1 for the accepted tiny model, and finite positive latency values. The script checks all logits with numpy.allclose(rtol=1e-5, atol=1e-6), plus agreement of predicted classes. Record actual numerical error and timing; do not require the historical latency to reproduce exactly.

If the Phase-1 ONNX file is absent, regenerate it into the new evidence directory with the following command, then use that file in the verifier. This checks reproducibility without overwriting the reference.

```
.\.venv\Scripts\python.exe model\export\export_simple_model.py `
  --output "$run\simple_model.onnx"
```

4. Run every implemented automated test

```
.\.venv\Scripts\python.exe -m unittest `
  tests.test_multi_exit_model tests.test_multi_exit_onnx -v
```

PASS: exit code 0, four tests run, and OK. Coverage: all four classifier output shapes; direct nonzero supervision to each head; rejection of mismatched loss weights; temporary four-output ONNX export, dynamic batch execution and PyTorch/ONNX numerical agreement. Legacy exporter deprecation warnings are known; distinguish warnings from failures.

5. Validate the accepted trained checkpoint on all data

```
.\.venv\Scripts\python.exe model\training\evaluate_multi_exit.py `
  --checkpoint model\artifacts\phase2-imagenette-5ep\best_final_exit.pt `
  --data-root data --workers 0 `
  --output "$run\validation_report.json"
```

Allow this full CPU evaluation to finish; do not substitute a smoke subset. Workers 0 avoids multiprocessing issues in agent sessions. The script may download the official Imagenette archive when missing; record network use and download failures. Load only the trusted project checkpoint because its loader uses weights_only=False.

5. Full-validation acceptance criteria

Require exit code 0, a new nonempty JSON report, checkpoint epoch 5, all four exit metrics and exactly 3,925 validation samples. Require the dataset archive SHA-256 below and class mapping consistent with checkpoint metadata.

```
64d0c4859f35a461889e0147755a999a48b49bf38a7e0f9bd27003f10db02fe5
```

Historical baseline: exit 1: 2,248 correct (57.27%), loss 1.378888; exit 2: 2,619 (66.73%), loss 1.053556; exit 3: 2,818 (71.80%), loss 0.875750; exit 4: 2,869 (73.10%), loss 0.829955. Weighted loss: 0.968464.

Compare fresh JSON with model/artifacts/phase2-imagenette-5ep/validation_report.json. For the same accepted checkpoint/data, require the correct counts to agree. Small floating-point loss differences may depend on device; report the difference and environment. If counts or provenance differ, investigate and do not claim reproduction. Record the actual device; CUDA may be selected automatically if available.

6. Re-export and verify the accepted four-exit ONNX model

```
.\.venv\Scripts\python.exe model\export\export_multi_exit.py `
  --checkpoint model\artifacts\phase2-imagenette-5ep\best_final_exit.pt `
  --data-root data `
  --output "$run\multi_exit_imagenette.onnx" `
  --report "$run\multi_exit.verification.json" `
  --verification-samples 10
```

PASS: exit code 0; fresh ONNX/report exist; ONNX checker and strict shape inference pass; opset 17; dynamic input [batch,3,128,128]; outputs exit_1_logits through exit_4_logits, each [batch,10]; every exit agrees with PyTorch at atol=1e-5 and rtol=1e-4. Inspect the report for ten samples spanning ten classes, checkpoint epoch/hash, dataset hash, graph structure and per-exit errors.

The accepted reference graph has 58 nodes and 48 initializers. Historical per-exit maximum errors were 2.3841858e-06, 9.5367432e-07, 1.4305115e-06 and 1.6689301e-06. Numerical tolerances determine success; byte-identical regenerated ONNX files are not required when provenance paths or exporter environments differ.

7. Verify reference artifact identity

```
Get-FileHash model\artifacts\phase3\multi_exit_imagenette.onnx `
  -Algorithm SHA256
Get-FileHash model\artifacts\phase2-imagenette-5ep\best_final_exit.pt `
  -Algorithm SHA256
```

Historical accepted ONNX SHA-256: 0cc08e571383279798c00019f0079bcbc107db49a1fe8efca444a4b754f35051. A difference means the artifact changed; inspect provenance before judging equivalence. Compare checkpoint hash to the existing Phase-3 report and fresh report. Never silently overwrite a changed reference.

8. Troubleshoot without hiding failures

C++ process exits immediately: run the executable directly with --model model/artifacts/simple_model.onnx --iterations 10 to expose stderr. Confirm the MSYS2 UCRT64 bin directory is on the process PATH and the ONNX Runtime SDK/DLL is present. The verifier captures subprocess output, so its exception alone may conceal the useful message.

Missing Python import: confirm the command uses .venv/Scripts/python.exe. Use the pinned requirements for recovery rather than arbitrary latest package upgrades. Record any local dependency changes.

Missing checkpoint or dataset: report the exact missing path. Obtain the accepted trusted checkpoint; do not replace validation with a newly trained model. Dataset download/hash errors block data-dependent stages, but model unit tests and the tiny C++ check can still run.

Four-output model rejected by C++: expected limitation. Phase-1 C++ supports the tiny single-output model; four-output C++ integration is the next milestone.

Different latency: repeat the 500-iteration parity command three times and report the range with hardware/load context. No fixed speed threshold exists. These timings cover the tiny inference call, not application throughput or adaptive-policy benefit.

Long full validation: monitor process/output and allow completion. A timeout or interrupted run is BLOCKED/incomplete, never PASS. Do not rerun training as a testing shortcut.

9. Close the run and deliver evidence

```
git status --short
Stop-Transcript
```

Write a final report in the new evidence folder. Include: date/time and timezone; Git commit and initial/final changes; OS/CPU/device and package versions; every command and exit code; stage result and reason; paths to transcript and JSON reports; parity errors; validation counts/accuracy/loss; artifact hashes; and any repairs or deviations. State explicitly whether all implemented stages passed or which remain blocked.

Demonstration for another person

Share your screen and run Stage 3 plus Stage 4 for a short live demonstration. Then show the fresh full-validation and ONNX verification reports. Explain that this proves the implemented inference/training/export pipeline; it does not demonstrate adaptive early exit. To let another machine repeat the run, provide the repository, this guide, trusted checkpoint/model artifacts and dependency setup. There is no browser URL or hosted dashboard today.

Evidence basis and update rule

This guide was derived from the local README, architecture/status records, tests/verify_phase1.py, the two unittest modules, model/training/evaluate_multi_exit.py, model/export/export_multi_exit.py and docs/phase2_validation.md and docs/phase3_validation.md. If code or milestones change, update commands and acceptance criteria from current source before testing.